

Semantic HTML

Write HTML That Carries Meaning

■■ SEMANTIC ELEMENTS

■ LEARNING OUTCOME

By the end of this module you will replace meaningless div-soup with semantic HTML5 elements, understand why semantics matter for accessibility and SEO, use ARIA attributes, and structure any page professionally.

■ Skills You Will Gain

- Use all HTML5 semantic elements: header, nav, main, article, section, aside, footer
- Understand why semantic HTML improves SEO and accessibility
- Add ARIA attributes for screen reader compatibility
- Use figure, figcaption, time, address, and mark elements
- Structure a blog post, news article, and portfolio page semantically
- Audit page semantics with Chrome DevTools Accessibility panel

■ Semantic HTML is the difference between a page that ranks on Google and one that doesn't. Screen readers, search engines, and other developers all rely on meaningful HTML structure.

SECTION 1

Why Semantics Matter

Three reasons semantic HTML is non-negotiable

1. ACCESSIBILITY: Screen readers use landmark elements (header, main, nav) for navigation. Users can jump directly to main content, skipping repeated navigation. 2. SEO: Search engines understand page structure from semantic elements. An h1 inside main carries more weight than text inside a div. 3. MAINTAINABILITY: Code is self-documenting. <article> tells every developer 'this is self-contained content' without reading a comment.

SECTION 2

Semantic Page Landmarks

Element	Semantic Meaning & Usage Rules
<header>	Introductory content for page or section. Contains logo, site title, navigation. Can appear inside article/section too.
<nav>	Primary navigation links. Use multiple nav elements if needed (main nav + breadcrumb). Add aria-label to distinguish them.
<main>	The main, unique content of the page. ONE per page. Skip link targets here. Never inside header/footer/aside.
<article>	Self-contained, independently distributable content. Blog posts, news, product cards, forum posts. Has its own heading.
<section>	Thematic grouping of content. Must have a heading (h2/h3). NOT a generic container — use div for that.
<aside>	Tangentially related content — sidebar, pull quotes, related articles, ads. Not essential to understanding main content.
<footer>	Footer for page or section. Copyright, links, contact info. Can appear inside article too.
<figure>	Self-contained media (image, chart, code) with optional caption. Use with figcaption.
<figcaption>	Caption for a figure element. Should describe or add value, not duplicate alt text.

SECTION 3

Complete Semantic Page Structure

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"> <meta name="viewport"
content="width=device-width, initial-scale=1.0"> <title>Tech Blog | Latest Articles</title> </head>
<body> <header> <a href="/" aria-label="Tech Blog Home">  </a>
<nav aria-label="Main navigation"> <ul> <li><a href="/articles"
aria-current="page">Articles</a></li> <li><a href="/tutorials">Tutorials</a></li> <li><a
```

```
href="/about">About</a></li> </ul> </nav> </header> <main> <article> <header> <h1>Understanding CSS
Grid Layout</h1> <p>By <address><a href="/authors/jane">Jane Doe</a></address> on <time
datetime="2024-01-15">January 15, 2024</time></p> </header> <section aria-label="Introduction">
<h2>What is CSS Grid?</h2> <p>CSS Grid is a two-dimensional layout system...</p> <figure>  <figcaption>Fig 1. A
basic 3x2 CSS Grid layout</figcaption> </figure> </section> <section aria-label="Basic syntax">
<h2>Getting Started</h2> <p>To enable Grid, apply <code>display: grid</code> to the <mark>container
element</mark>, not the items.</p> </section> </article> <aside aria-label="Related articles">
<h2>You Might Also Like</h2> <ul> <li><a href="/flexbox">Mastering Flexbox</a></li> <li><a
href="/responsive">Responsive Design</a></li> </ul> </aside> </main> <footer> <p>&copy; 2024 Tech
Blog. <a href="/privacy">Privacy Policy</a></p> </footer> </body> </html>
```

SECTION 4

ARIA Attributes for Accessibility

ARIA Attribute	Purpose & Example
aria-label	Provides an accessible name when visible text is absent. <nav aria-label='Breadcrumb'>
aria-labelledby	Points to another element's id as the label. <section aria-labelledby='sec1-heading'>
aria-describedby	Points to a description element. <input aria-describedby='pwd-requirements'>
aria-current	Marks the active item in navigation.
aria-hidden	Hides decorative element from screen readers. ★
aria-expanded	State of expandable widgets. <button aria-expanded='false'>Menu</button>
aria-required	Marks a required form field. <input aria-required='true'>
role='alert'	Announces dynamic content to screen readers immediately on change

SECTION 5

Semantic Text Elements

Element	Meaning	Example Use
<time datetime>	Machine-readable date/time	<time datetime='2024-01-15'>Jan 15</time>
<address>	Contact information for nearest article/body	Author contact, company address
<mark>	Highlighted/relevant text	Search results highlight matching term
<code>	Inline code snippet	Use the <code>display: flex</code> property
<pre>	Preformatted text (preserves whitespace)	Multi-line code blocks
<abbr title>	Abbreviation with expansion	<abbr title='HyperText Markup Language'>HTML</abbr>
<cite>	Title of creative work	<cite>The Art of War</cite>

<code><blockquote></code>	Extended quotation from another source	Quotes with cite attribute for source
<code><details>/<summary></code>	Disclosure widget — expand/collapse	FAQ sections, spoilers

SECTION 6

Div Soup vs Semantic HTML

```
<!-- ■ DIV SOUP – no meaning, hard to maintain --> <div class="top"> <div class="logo"></div> <div class="menu"> <div class="menu-item"><a
href="/">Home</a></div> </div> </div> <div class="content"> <div class="post"> <div
class="post-title">My Article</div> <div class="post-text">Some content here...</div> </div> </div>
<div class="bottom"><div class="copy">&copy; 2024</div></div> <!-- ■ SEMANTIC – self-documenting,
accessible, SEO-friendly --> <header>  <nav> <a
href="/">Home</a> </nav> </header> <main> <article> <h1>My Article</h1> <p>Some content here...</p>
</article> </main> <footer><p>&copy; 2024</p></footer>
```

■ PRACTICE Q & A

- Q1.** What is the difference between section and div?
- A: section has semantic meaning — it's a thematic grouping of content with a heading. div has NO semantic meaning — it's a generic container used for CSS layout only. Use section when the content forms a distinct theme; use div when you just need a styling hook.*
- Q2.** Can you have more than one nav element on a page?
- A: Yes — you can have multiple nav elements (main navigation, breadcrumb, footer links). Add aria-label to each to distinguish them: <nav aria-label='Main navigation'> and <nav aria-label='Breadcrumb'>.*
- Q3.** What is the difference between article and section?
- A: article: self-contained content that makes sense standalone and could be syndicated (blog post, news item, product review). section: thematic grouping within a larger context — doesn't make sense extracted from the page.*
- Q4.** What does aria-hidden='true' do and when should you use it?
- A: Hides the element from assistive technologies (screen readers) while keeping it visually visible. Use on decorative icons, duplicate content, or elements that would create confusion if announced: ★.*
- Q5.** Why is using an h2 inside an article's header better than a div with a class?
- A: Heading elements (h1-h6) create the document outline — screen reader users navigate pages by jumping between headings. A div with class='title' is invisible to assistive technology and search engines.*

■■ Semantic HTML Cheat Sheet	
<code><header></code>	Site or section header
<code><nav aria-label></code>	Navigation — label each one
<code><main></code>	One per page — unique content
<code><article></code>	Self-contained, distributable
<code><section></code>	Thematic group — needs heading
<code><aside></code>	Related but non-essential content
<code><footer></code>	Page or section footer
<code><figure>+<figcaption></code>	Media with caption
<code><time datetime></code>	Machine-readable date
<code><mark></code>	Highlighted/relevant text

aria-label	Name for landmark without visible text
aria-current='page'	Mark active nav link